

FORGE — CURRENT PROJECT STATUS & PARTNER HANDOFF

Robust and Selective Knowledge Unlearning in Large Language Models

Framework: FORGE | Repository: Mihiiikaaaa/Forge | Working branch: mihika

1. Purpose

This document is the single handoff/status reference for the second contributor. It records what has been built, what is provisional, what depends on the trained model, exactly what the GPU-training contributor must deliver, how the checkpoint connects to the existing pipeline, and what remains before publication-quality experiments.

2. Research objective

FORGE investigates whether targeted LLM unlearning genuinely removes target knowledge or mainly suppresses its expression. The desired behavior is selective forgetting: target knowledge is forgotten while closely related, moderately related, and unrelated knowledge are preserved as much as possible.

Dimension	Research question
Target forgetting	Does the model stop assigning high probability to the forgotten answer?
Paraphrase robustness	Does forgetting survive different formulations of the same question?
Indirect inference	Can the model reconstruct the forgotten fact from indirect clues?
Retention	Does unrelated/retained knowledge remain usable?
Blast radius	How far does collateral damage extend beyond the target?
Relearning resistance	Does forgotten knowledge quickly return after brief retraining?
Trade-off	How much forgetting is obtained for how much utility/collateral damage?

Research-order rule: the final FORGE unlearning algorithm is intentionally not fixed yet. The sequence is learned model → baseline unlearning → robust evaluation → failure analysis → proposed method → ablations → final comparison.

3. Current status

Area	Status	Details
Git/repository	DONE	Git repo and mihika working branch established.
Python environment	DONE	Project .venv and core dependencies set up.
TOFU loading	DONE	Current environment successfully loads 4,000 training examples.
Dataset preparation	DONE	1%, 5%, 10% deterministic forget/retain partitions generated.
Base model sanity check	DONE	TinyLlama-1.1B loads and inference works.
Initial evaluation	DONE/PARTIAL	Generation-based sanity evaluator exists; not final quantitative scoring.
Metrics	BUILT/PROVISIONAL	Helper functions exist; final definitions must be aligned with NLL/utility metrics.
Likelihood evaluation	BUILT/BLOCKED	Answer-level NLL direction exists; needs learned checkpoint to execute.
Paraphrase infrastructure	BUILT/TEMPLATE	40 targets prepared; actual paraphrases are not populated.
Blast-radius infrastructure	BUILT/TEMPLATE	40 target records prepared; relatedness candidates are not populated.
Learned TOFU model	BLOCKER	Proper checkpoint is not yet available.
Baseline unlearning	PENDING	Cannot meaningfully run until target

		knowledge is learned.
Final FORGE method	INTENTIONALLY PENDING	Should follow empirical failure analysis.
Final experiments/paper	PENDING	Depend on the above.

4. Exact file inventory

File/component	State	Purpose	Later change?
config/base.yaml	BUILT	Central dataset/model/training/evaluation/path configuration.	YES
src/data/load_tofu.py	DONE	Loads TOFU and prints structure/sample.	Mostly no
src/data/prepare_tofu.py	DONE	Creates 1%, 5%, 10% forget/retain partitions.	YES: verify official protocol
src/evaluation/metrics.py	BUILT	Forgetting/retention/drop helpers.	YES: finalize around primary metric
src/evaluation/evaluation_utils.py	DONE	JSON save/load utilities.	Probably no
src/evaluation/evaluate_baseline.py	PARTIAL/TEMP	Qualitative generation sanity check.	YES: not final metric
src/evaluation/evaluate_forgetting.py	PARTIAL/TEMP	Pre/post-style generation inspection scaffold.	YES: integrate with quantitative scoring
src/evaluation/answer_likelihood.py	BUILT/VERIFY	Answer-token NLL conditioned on question.	YES: verify masking/token alignment
src/evaluation/evaluate_likelihood.py	BUILT/BLOCKED	Pre-unlearning quantitative likelihood evaluator.	YES: run/validate after checkpoint
src/evaluation/robustness/paraphrases.py	BUILT/TEMPLATE	Creates paraphrase JSON skeleton.	YES: populate
src/evaluation/robustness/validate_paraphrases.py	BUILT	Validates paraphrase entries.	YES: extend final schema
src/evaluation/blast_radius/build_blast_radius.py	BUILT/TEMPLATE	Creates blast_radius.json skeleton for 40 targets.	YES: populate
src/evaluation/blast_radius/semantic_distance.py	BUILT/UTILITY	MiniLM cosine-similarity helper.	YES: integrate
data/processed/forget_1 + retain_1	GENERATED	1% experiment partitions: 40/3960.	Only if protocol changes
data/processed/forget_5 + retain_5	GENERATED	5% experiment partitions: 200/3800.	Only if protocol changes
data/processed/forget_10 + retain_10	GENERATED	10% experiment partitions: 400/3600.	Only if protocol changes
dataset_statistics.json	GENERATED	Split statistics.	Update if protocol changes
forget_paraphrases.json	TEMPLATE	40 records with empty paraphrase arrays.	YES
blast_radius.json	TEMPLATE	40 records with empty radius categories.	YES

5. Half-working / temporary components — read this carefully

- evaluate_baseline.py and evaluate_forgetting.py are qualitative sanity tools. Their generated answers must not be presented as the final quantitative forgetting metric.
- answer_likelihood.py/evaluate_likelihood.py are the intended quantitative direction, but they must be tested on the real learned checkpoint and the answer-only masking/tokenization must be verified.
- metrics.py contains provisional generic helpers. Do not treat their current outputs as final paper metrics until the NLL/utility definitions are frozen.
- prepare_tofu.py currently uses the project's deterministic custom partitions. Before publication, verify them against the official TOFU experimental protocol and revise if necessary.
- paraphrases.py only creates empty slots; 160 validated paraphrases do not yet exist.
- build_blast_radius.py only creates a template; the blast-radius dataset is not complete.
- semantic_distance.py is only a utility. Do not impose arbitrary similarity thresholds without validation.
- The final unlearning algorithm is intentionally absent; this is a research-design decision.

6. Timeline position

The project has completed the setup, data, base-model sanity, and initial evaluation scaffolding stages and is now at the transition into heavy learned-model training and serious evaluation. The shared 84-day plan should be used as the master schedule; this document deliberately does not invent a calendar day/date beyond the verified milestones.

Timeline block	State	Approx. completion
Setup/Git/environment	Completed	100%
TOFU + partitions	Completed	100%
Base-model sanity	Completed	100%
Evaluation scaffolding	Completed as scaffold; metric finalization pending	~80%
Robustness layer	Scaffolded; data/evaluators pending	~50%
Learned-model training	Partner task/current blocker	0% final checkpoint
Baseline unlearning	Pending	0%
Failure diagnosis + FORGE method	Pending by design	0%
Ablations/final experiments	Pending	0%
Paper finalization	Pending	0%

7. Partner's exact GPU-training responsibility

The partner must train a model that has actually learned the eventual forget knowledge. A prior local attempt trained only on retain_1 and stalled on the Mac; that is not a valid foundation for unlearning the forget set.

1. Pull the latest project code/config and preserve the existing evaluation infrastructure.
2. Train a proper learned TinyLlama checkpoint using the intended TOFU training data/protocol so the target knowledge is represented before unlearning.
3. Do not train only on retain_1 when retain_1 excludes the target facts we later intend to forget.
4. Record seed, model version, dataset/source/version, learning rate, batch size, gradient accumulation, epochs, tokenizer settings, GPU, CUDA/PyTorch/Transformers versions, and training time.
5. Save a Hugging Face-compatible model + tokenizer as models/tofu_learned, or provide an agreed external model path if the weights cannot be transferred conveniently.
6. Run a sanity check demonstrating meaningful target knowledge before unlearning.
7. Run python src/evaluation/evaluate_likelihood.py and preserve results/raw/baseline/pre_unlearning_likelihood.json.
8. Do not overwrite the learned checkpoint with an unlearned checkpoint.
9. Do not push large model weights to normal Git unless the team deliberately uses Git LFS/approved model storage.

8. Partner deliverables

Deliverable	Evidence	Acceptance condition
Learned checkpoint	models/tofu_learned or agreed storage	Loads with AutoTokenizer/AutoModelForCausalLM.
Training record	Log/config	Completed run + exact settings recorded.
Learning sanity check	Sample outputs and/or NLL	Target knowledge is measurably present.
Pre-unlearning likelihood	pre_unlearning_likelihood.json	Forget + retain records exist.
Hardware/software record	Experiment note	GPU/software versions recorded.
Git handoff	Branch/commit	Code changes are pushed and reproducible.

9. How everything connects after training

The intended research pipeline is:

TOFU → learned checkpoint → pre-unlearning NLL → baseline unlearning → post-unlearning NLL → paraphrase/indirect/blast-radius/retention/relearning evaluation → statistics/plots → failure analysis → FORGE method → ablation → final comparison.

Stage	Existing / future component	Input	Expected output
Train	Partner training pipeline	TOFU	models/tofu_learned

Load/quantify	evaluate_likelihood.py + answer_likelihood.py	learned model + Q/A	answer NLL
Pre-unlearning freeze	evaluation results	forget + retain	pre_unlearning_likelihood.json
Baseline unlearning	TO BUILD	learned model + forget/retain	baseline-unlearned checkpoint
Post-unlearning	TO BUILD/INTEGRATE	unlearned checkpoint	post-unlearning NLL
Paraphrases	TO BUILD	paraphrase JSON + checkpoint	robust forgetting
Indirect inference	TO BUILD	indirect questions + checkpoint	inference leakage
Blast radius	TO BUILD	TOFU candidates + checkpoint	damage vs semantic distance
Relearning	TO BUILD	unlearned model + forget data	relearning curve
Analysis	TO BUILD	all JSON results	tables/plots/statistics

10. Exact handoff after the model is trained

- Partner gives the branch/commit hash and checkpoint location.
- Verify the checkpoint loads before doing any unlearning.
- Run: `python src/evaluation/evaluate_likelihood.py`
- Inspect results/raw/baseline/pre_unlearning_likelihood.json.
- Confirm both forget and retain results exist and that target knowledge is actually represented.
- Freeze this checkpoint as the PRE-UNLEARNING reference. Never overwrite it.
- Implement/run the chosen baseline unlearning method on a copy of the learned model.
- Evaluate the resulting checkpoint using the same before/after metric setup.
- Then execute robustness, blast-radius, retention, and relearning experiments.
- Only after failure analysis should the final FORGE method be fixed.

11. Files to verify and expected outputs

File/check	Command/action	Expected evidence
config/base.yaml	<code>cat config/base.yaml</code>	Correct model/dataset/training/path settings.
load_tofu.py	<code>python src/data/load_tofu.py</code>	TOFU loads; current environment: 4,000 train rows.
prepare_tofu.py	<code>python src/data/prepare_tofu.py</code>	1%=40, 5%=200, 10%=400 forget examples.
Checkpoint	AutoTokenizer/AutoModel load test	CHECKPOINT LOAD OK.
evaluate_likelihood.py	<code>python src/evaluation/evaluate_likelihood.py</code>	JSON containing forget + retain NLL.
metrics.py	Manual review	Definitions consistent with final metric.
evaluate_forgetting.py	Optional sanity run	Qualitative generation only.
validate_paraphrases.py	<code>python src/evaluation/robustness/validate_paraphrases.py</code>	All required paraphrase entries pass.
blast_radius.json	Schema/content validation	Target + related + moderate + unrelated entries populated.

12. Work Mihika can do while partner trains

- Populate/quality-check $40 \times 4 = 160$ paraphrases.
- Build semantic candidate discovery for all 40 forget targets.
- Inspect and classify blast-radius candidates.
- Integrate semantic similarity as a continuous variable.
- Build the final paraphrase evaluator using the same quantitative metric.
- Build indirect-inference dataset/evaluator.
- Build relearning experiment scaffold.
- Build result aggregation, confidence intervals/statistics, and publication-quality plots.
- Continue literature/gap matrix; let observed baseline failures determine the final FORGE method.

13. State immediately after partner's part

Component	Expected state
TOFU pipeline	Verified/reproducible.
Learned model	Verified TinyLlama checkpoint that learned target TOFU knowledge.
Pre-unlearning evaluation	Forget/retain answer-NLL JSON saved.
Robustness infrastructure	Ready for model execution; ideally datasets populated.
Baseline unlearning	Next major experimental step unless explicitly delegated to partner.
FORGE method	Still pending empirical failure analysis.
Final paper results	Not yet available.

14. Deliverables already achieved

- Git repository + working branch.
- Project structure + isolated .venv.
- Central configuration.
- TOFU loader.
- 1%, 5%, 10% forget/retain partitions.
- Dataset statistics.
- TinyLlama loading/sanity check.
- Initial qualitative evaluation.
- Metric helpers.
- JSON result utilities.
- Pre-unlearning generation scaffold.
- Answer-level NLL utility and evaluator scaffold.
- Paraphrase template + validator.
- Blast-radius template.
- Semantic-distance utility.

15. Deliverables after partner's GPU phase

- Reproducible learned checkpoint.
- Training configuration/log and hardware/software record.
- Checkpoint load verification.
- Evidence target knowledge was learned.
- Pre-unlearning likelihood results.
- Frozen PRE-UNLEARNING reference checkpoint.
- Clear Git/experiment handoff.

16. Final research deliverables

- Strong baseline unlearning implementations.
- Direct forgetting + retention results.
- Paraphrase robustness results.
- Indirect-inference results.
- Relearning-resistance curves.
- Blast-radius/semantic-distance analysis.
- 1%, 5%, 10% forget-size experiments.
- Baseline comparison and ablations.
- Statistical analysis and confidence intervals.
- Publication-quality tables/figures.
- Evidence-based FORGE method.

- Final paper, code, logs, and reproducibility documentation.

17. Partner checklist

- ☐ Pull latest code.
- ☐ Confirm config and data partitions.
- ☐ Train a model that has learned the eventual forget knowledge.
- ☐ Record exact training/hardware/software settings.
- ☐ Save tokenizer + model as models/tofu_learned or agreed external storage.
- ☐ Verify checkpoint loads.
- ☐ Run target-knowledge sanity check.
- ☐ Run python src/evaluation/evaluate_likelihood.py.
- ☐ Confirm results/raw/baseline/pre_unlearning_likelihood.json exists.
- ☐ Send commit hash + checkpoint location + training log.
- ☐ Keep PRE-UNLEARNING checkpoint separate from all unlearned checkpoints.
- ☐ Do not claim final forgetting success yet.

18. Bottom line

Current state: the core data, environment, model sanity, evaluation scaffolding, likelihood direction, paraphrase infrastructure, and blast-radius infrastructure are built. The major blocker is the properly learned TOFU checkpoint. After the partner delivers it, the project reaches a reproducible PRE-UNLEARNING milestone. The next research phase is baseline unlearning, robust evaluation, collateral-damage/relearning analysis, failure diagnosis, then the evidence-driven FORGE method and final publication experiments.